Features of ActiveMQ

There is a long list of features on the ActiveMQ website, but I will highlight and discuss the key ones:

- Cross-language compliance and compatibility: This is an important feature, since applications may be written in different languages like C/C++, Python, Ruby, Java, etc. Allowing cross-language compatibility allows communication between very diverse applications, and allows seamless integration. This is probably the most important feature for message-oriented middleware.
- AJAX and REST support: Web services and applications can easily communicate, since AJAX and REST are part of the core of Web services and applications out there.
- Compatible with various transport protocols: Important because messages can be transferred effectively only if there is a wide variety of protocols for transport. Examples of transport protocols include TCP (Transmission Control Protocol), UDP (User Datagram Protocol) and SSL (Secure Socket Layer).
- Support for persistence: In computing terms, persistence is the capability of an application to preserve its state. In other words, it's the ability of the application to remember. This is done by the program recording what it needs to remember in text files or databases. In the context of ActiveMQ, persistence is used mainly for the database-linked persistence of applications.
- High-performance computing and high availability: In simple words, this is nothing but ensuring that any form of failure doesn't affect the user. This is achieved mainly through clustering. Here, many instances of the application (either on the same machine, but usually spread across multiple physical machines) are running, in such a way that if one instance fails, another one takes over. This mission-critical feature is a big plus in ActiveMQ.

Usage scenario

The product is great, sounds completely robust and the features are fine and dandy, but where is it used? I will demonstrate a scenario where ActiveMQ (or usually any message-broking middleware) would work. For the sake of an explanation, I will cite a specific scenario where ActiveMQ behaves like a message bus—or, more specifically, an enterprise service bus. Figure 1 is a diagram that will explain it.

Other popular open source middleware

Apart from ActiveMQ, the world of enterprise computing is full of other open source middleware. Here are some examples, with a short description of each.

JDBC: The famous Java DataBase Connectivity API is an SQL-Oriented Data Access middleware. It allows Java programs to have database access, and thus be persistent. Another similar one is ODBC (Open DataBase Connectivity).

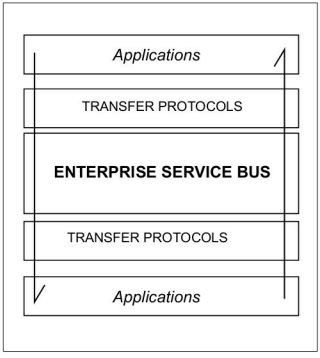

Figure 1: The Enterprise Service Bus (ESB) concept

CORBA: Developed by the Object Management Group (OMG), the Common Object Request Broker Architecture is a class of middleware that fits the object-broker type. Essentially, it allows communication between many applications and software components to exchange objects, even when they are written in different languages and are running on different computers. Note that CORBA is a standard, and not middleware, per se. A popular open source implementation of CORBA is MICO (MICO is a recursive acronym for Mico Is COrba).

This is an introduction to the concept of middleware. It goes way deeper than the scope of this article, but hopefully what you read here will help you as a starting point. Happy learning! END

Links

[1] Judith Hurwitz's 'Sorting Out Middleware'—http://web.archive.org/web/20061017153344/http:www.dbmsmag.com/9801d04.html
[2.] Apache ActiveMQ Home—http://activemq.apache.org/
[3.] The CORBA Standard Specification—http://www.omg.org/spec/CORBA/
[4.] MICO—http://www.mico.org

By: Siddharth Mankad

The author is a new media designer, with an affinity for physical computing, computer software and code, apart from country music.